FINAL YEAR CAPSTONE ENGINEERING PROJECT DEFENSE

AAROGYA: Multi-Agent Embodied AI Robot for Autonomous Hospital Assistance

Vision-Language Spatial Reasoning, Dynamic SLAM Exploration, Clinical Safety Workflows & PhysX Digital Twin

PROJECT METADATA & COMMITTEE SPECIFICATION

Domain: Autonomous Systems, Embodied AI, Healthcare Robotics | Frameworks: FastAPI, ROS 2 Humble, NVIDIA Isaac Sim 5.1

Verified Test Suite: 178 Automated Tests Passing (100% Green) | Architecture: FIPA Contract Net Protocol + 2D Log-Odds SLAM

Clinical Motivation & Real-World Hospital Bottlenecks

1. Nursing Transit Fatigue

Nurses spend up to 35% of 12-hour shifts walking long corridors for routine tasks (delivering stat meds, blood gas samples, linens). This causes physical exhaustion, burnout, and detracts from direct patient bedside care.

2. Medication Delivery Errors

Manual medication transit in overcrowded wards introduces documentation gaps and duplicate delivery hazards. Lack of strict bedside identity verification risks adverse drug events (ADEs).

3. Emergency Priority Gaps

During 'Code Blue' cardiac emergencies, critical crash carts and defibrillators are delayed by corridor traffic. Routine hospital transport lacks autonomous priority preemption mechanisms.

4. Dynamic Obstacle Chokepoints

Hospital corridors are unpredictable: gurneys, IV poles, wheelchairs, and visitors constantly block pathways. Static pathing AGVs deadlock; robots must adapt dynamically.

Formal Problem Statement & The Multi-Layer Integration Challenge

FORMAL PROBLEM FORMULATION

Develop an embodied multi-agent robotic system capable of: (1) parsing unconstrained natural clinical voice commands, (2) coordinating a heterogeneous AMR fleet under clinical priority constraints, (3) dynamically mapping and navigating an unpredictable hospital digital twin in NVIDIA Isaac Sim without synthetic telemetry, and (4) enforcing end-to-end clinical role authorization and ACID audit compliance.

The HRI & Reasoning Challenge

- Clinical voice dictation is noisy and context-heavy.
- Must extract Patient ID, Room, Medication, Priority.
- Requires role validation (Doctors vs Nurses vs Visitors).
- Must reject unauthorized or clinical-risk requests.

The Multi-Agent Dispatch Challenge

- AMRs have distinct roles (Delivery, Emergency, Guide).
- Fixed heuristics fail when batteries deplete or paths block.
- Requires decentralized bidding (Contract Net Protocol).
- Priority-1 Code Blue emergencies must preempt active tasks.

The Embodied SLAM Challenge

- Corridors feature dynamic gurneys and human crowds.
- Must build live 2D log-odds occupancy grids from LiDAR.
- Autonomous exploration via frontier clustering.
- Zero pre-baked paths or fake UI animation allowed.

Project Objectives & Verifiable Technical Deliverables

Objective 1: Embodied Digital Twin
Deploy an 80m x 60m realistic hospital environment in NVIDIA Isaac Sim 5.1 with PhysX 5.1 dynamics, colliders, and 4 heterogeneous differential-drive AMRs. ➔ DELIVERABLE: USD Hospital Stage with 18 distinct rooms and active LiDAR raycasters.

Objective 2: Natural Clinical HRI
Implement natural voice dictation via Web Speech API and backend NLU regex/semantic parsers with PyJWT Role-Based Access Control. ➔ DELIVERABLE: Fast, client-side dictation + RBAC endpoint blocking unauthorized requests.

Objective 3: Multi-Agent Coordinator
Design and verify a FIPA-compliant Contract Net bidding engine for task allocation with dynamic preemption for Code Blue emergencies. ➔ DELIVERABLE: CoordinatorAgent with bidding evaluation across battery, distance, and role.

Objective 4: Autonomous SLAM Exploration
Develop a 2D Log-Odds occupancy mapping engine and frontier cluster detector that autonomously maps hospital corridors without synthetic data. ➔ DELIVERABLE: SLAMExplorationEngine with Bresenham raycasting and BFS frontier extraction.

Objective 5: Clinical Safety Audit Engine
Enforce duplicate delivery prevention, bedside verification locks, and SQLite ACID audit trail logging for hospital compliance. ➔ DELIVERABLE: TaskEngine duplicate validation + immutable audit log table.

Existing Approaches vs. AAROGYA Platform (Architectural Matrix)

Technical Dimension	Commercial AGVs (Aethon TUG)	Academic ROS Nav Stacks	AAROGYA Embodied Platform
Task Allocation	Centralized FIFO Dispatcher	Static ROS 2 action goal queue	**Decentralized FIPA Contract Net with role suitability**
Emergency Preemption	None; tasks queued sequentially	Manual topic override required	**Automated Priority-1 Code Blue preemption stack**
Clinical HRI & Audio	Fixed touchscreens on chassis	No natural language processing	**Web Speech API voice dictation + RBAC token guard**
Corridor SLAM Exploration	Pre-recorded CAD map only	SLAM Toolbox without frontier logic	**Dynamic 2D log-odds SLAM with BFS frontier clustering**
Safety & Audit Integrity	Proprietary database, closed logs	Ephemeral rosbag files	**ACID SQLite models with immutable clinical audit trail**

Proposed System: End-to-End Embodied Information Flow

1. Clinical Input

Doctor voice dictation or UI task submission with JWT role claim.

2. Backend Ingestion

FastAPI endpoint validates RBAC permissions and clinical rules.

3. Semantic Parsing

MissionUnderstander maps intent, destination, and patient ID.

4. Duplicate Lock

TaskEngine checks active inpatient tasks to prevent double dispensing.

5. Multi-Agent Bidding

CoordinatorAgent solicits bids; evaluates battery, distance, and role.

6. Mission Dispatch

Route constructed; dispatched via ROS 2 & HTTP Port 8080.

7. PhysX Navigation

Isaac Sim AMR engages differential drive; CollisionGuard slows near obstacles.

8. SLAM Update

360° LiDAR raycasting updates live log-odds occupancy grid at 10 Hz.

PART 18 ARCHITECTURE BLOCK DIAGRAM

DIAGRAM 1: High-Level End-to-End AAROGYA System Architecture

1. HOSPITAL CLINICAL STAKEHOLDERS
Doctors, Charge Nurses, Ward Pharmacists, Hospital Admins | Voice Audio Dictation, Browser REST Actions, Dynamic Clinical Web UI

▼ [Browser Web Speech Audio / HTTPS JSON Payloads / Secure WSS Channels]

2. AAROGYA HRI PRESENTATION LAYER
Single Page Application (frontend/index.html, static/js/app.js) | Dual-Canvas Viewports (CAD Blueprint vs Live SLAM), Voice Bar, Status HUD

▼ [Async REST Endpoints: /api/voice_command, /api/missions, /api/simulation/explore]

3. FASTAPI BACKEND GATEWAY & CLINICAL WORKFLOW
REST/ASGI Services (backend/main.py, task_engine.py, models.py) | PyJWT Role Guard, Duplicate Delivery Lock, SQLite WAL Mode Audit Logs

▼ [Parsed Intent Tuple (INTENT, DESTINATION, PRIORITY) & Clinical Task Ticket]

4. AI / NLU & MULTI-AGENT ORCHESTRATION LAYER
Multi-Agent System (agents/coordinator_agent.py, nlu_engine.py) | CoordinatorAgent, FIPA Contract Net Protocol Bidding, Priority-1 Preemption

▼ [Dispatched Mission Payload: winner_robot_id, Goal (gx, gy), Priority Class]

5. MISSION PLANNING & SPATIAL DISPATCH
Spatial Engine (navigation/route_planner.py, collision_guard.py) | A* Global Pathfinding, BFS Frontier Clustering, Dynamic Hazard Scoring

▼ [ROS 2 Message Bus & HTTP JSON IPC (Port 8080 /api/dispatch_mission)]

6. ROS 2 HUMBLE & SENSOR FUSION MIDDLEWARE
Robotics Layer (rclpy, sensor_msgs, nav_msgs, geometry_msgs) | Topics: /scan (LiDAR), /odom (Odometry), /tf & /tf_static, /cmd_vel (Twist)

▼ [PhysX 5.1 Actuator Joint Torques (v, ω) & Planar Raycast Sensor Returns]

7. NVIDIA ISAAC SIM 5.1 DIGITAL TWIN RUNTIME
Simulation Stack (isaac_sim_hospital/scripts/run_embodied_hospital_ai.py) | 80m x 60m USD Stage, PhysX 5.1 60Hz, 4 AMRs, Dynamic Obstacles, Port 8080

DECOUPLED ARCHITECTURE & FAULT DOMAINS

- Isolated Failure Boundaries: The backend, multi-agent engine, and database run independently of the simulation.
- Simulator Standby Guard: If Isaac Sim is offline, the API reports SIMULATOR_STANDBY rather than faking execution.
- State Synchronization: FastAPI SimulationOrchestrator syncs at 10 Hz non-blocking over Port 8080.

CONCRETE PORT BINDINGS & NETWORK TOPOLOGY

- Port 8000 (FastAPI): Clinical REST APIs, WebSockets (/ws/telemetry), Static UI.
- Port 8080 (Perception Server): LiDAR raycast ingestion, POV camera MJPEG stream, mission dispatch.
- Port 8081 (Database Server): Hospital spatial layout & dynamic obstacle state synchronization.
- ROS 2 Domain ID: Standard DDS topics /scan, /odom, /tf, /cmd_vel for robot control.

END-TO-END TELEMETRY & LATENCY SPECIFICATION

- Voice Dictation Latency: ~120 ms client-side audio capture to intent token extraction.
- Contract Net Bidding: < 5 ms to solicit bids across 4 AMRs and assign winner.
- Dispatch-to-Actuation: < 15 ms from HTTP POST to PhysX differential wheel torque.
- Full Telemetry Refresh: 10 Hz continuous state broadcast without UI freeze.

AAROGYA CODE REVIEW DEFENSE

End-to-End Task Lifecycle: Natural Request to Physical Execution

Step 1: Clinical Dictation
Doctor dictating: 'Deliver Metoprolol to Arun Patel in ICU Bed 3'. Captured via browser Web Speech API.

Step 2: RBAC Role Verification
FastAPI verifies JWT signature. Checks that role == DOCTOR or NURSE. Unauthorized visitor requests are denied with 403.

Step 3: Semantic NLU Parsing
MissionUnderstander regex/semantic engine maps natural tokens: Patient=P001, Room=ICU-01, Med=Metoprolol.

Step 4: ACID DB Duplicate Lock
TaskEngine queries SQLite database. Ensures no conflicting pending/in-progress delivery exists for Patient P001.

Step 5: Contract Net Auction
CoordinatorAgent solicits bids from 4 AMRs. Evaluates distance, battery, and role suitability. Awards mission to Arogya_Delivery.

Step 6: Route & Mission Dispatch
Route planner constructs 2-phase route: Pickup at Pharmacy (PHM-01) -> Bedside ICU-01. Dispatched via Port 8080.

Step 7: Isaac Sim Physical Transit
Arogya_Delivery steps PhysX differential drive wheels, integrates 360° LiDAR, navigates around obstacles in real-time.

Step 8: Bedside Verification & Audit
Robot confirms bedside NFC tag at ICU-01. Medicine container unlocks. Timestamped immutable audit log saved.

Comprehensive Technology Stack & Implementation Standards

Core Application & Backend

- **Runtime:**
  Python 3.13.3 (Standard Library + AsyncIO)
- **Web Framework:**
  FastAPI 0.115 + Starlette ASGI
- **Database ORM:**
  SQLAlchemy 2.0.38 + SQLite 3 (WAL Mode)
- **Security & Tokens:**
  PyJWT 2.10.1 (HMAC-SHA256 Signatures)
- **Network Client:**
  HTTPX 0.28.1 (Asynchronous REST I/O)

Multi-Agent & Robotics

- **Coordination:**
  FIPA Contract Net Protocol (Decentralized)
- **Robotics Middleware:**
  ROS 2 Humble Hawksbill (rclpy, nav2_msgs)
- **Simulation Digital Twin:**
  NVIDIA Isaac Sim 5.1 (Omniverse Kit, PhysX 5.1)
- **Sensor Modeling:**
  360° Planar LiDAR (180 rays @ 10 Hz)
- **Kinematics:**
  Non-Holonomic Differential Drive AMR

Frontend & HRI Presentation

- **Presentation Layer:**
  HTML5 + Vanilla CSS + JavaScript (ES6 Modules)
- **Audio Capture:**
  Web Speech API (Real-Time Voice Dictation)
- **Spatial Viewport:**
  Dual HTML5 Canvas (CAD Map vs SLAM Occupancy)
- **Live Telemetry:**
  WebSocket (10 Hz Full-Duplex State Sync)
- **Video Streaming:**
  MJPEG Video Stream (RGB Camera Perception)

Code Review: Repository Inventory & Backend Dispatch Engine

REPOSITORY STRUCTURE & LINES OF CODE

```
HospitalRobotNav/
├── backend/                    # 1,190+ Lines (FastAPI REST &
WebSockets)
│   ├── main.py                 # 60+ Endpoints, Orchestrator, Dispatch
│   ├── task_engine.py          # Clinical task state machine &
duplicate locks
│   ├── voice_service.py        # Voice dispatch & intent classification
│   └── auth.py                 # PyJWT role verification & token
generation
├── database/                   # SQLAlchemy ORM Models & SQLite Session
│   ├── models.py               # Relational schema (Patient, Robot,
Task, Audit)
│   └── session.py              # WAL mode connection factory
├── agents/                     # Multi-Agent Coordination Engine
│   ├── coordinator_agent.py    # FIPA Contract Net auctioneer
│   ├── delivery_agent.py       # AMR mission execution controller
│   └── nlu_engine.py           # Semantic voice-to-mission parser
├── navigation/                 # Spatial SLAM & Route Planning
│   ├── slam_exploration_engine.py # 2D Log-Odds SLAM & Frontier
Clustering
│   ├── route_planner.py        # A* global pathfinding with clearance
│   └── collision_guard.py      # Social deceleration & dynamic
avoidance
├── isaac_sim_hospital/         # NVIDIA Isaac Sim 5.1 Digital Twin
│   ├── scripts/run_embodied_hospital_ai.py # PhysX main loop
│   └── web/perception_server.py # 10 Hz HTTP bridge (Port 8080)
└── tests/                      # 178 Automated Tests (29 Suites, 100%
Green)
```

FILE: backend/main.py

```
# Dynamic Corridor Obstacle Injection (Lines 898-940)
@app.post('/api/simulation/toggle_obstacle')
def toggle_dynamic_obstacle(req: ObstacleToggleRequest):
    active = SLAM_ENGINE.get_dynamic_obstacles()
    existing = [o for o in active if o['id'] == 'DYN-OBS-01']
    if existing:
        SLAM_ENGINE.clear_dynamic_obstacle('DYN-OBS-01')
        status_msg = 'CLEARED'
    else:
        SLAM_ENGINE.detect_dynamic_obstacle('DYN-OBS-01', req.x,
req.y)
        status_msg = 'INJECTED'
    # Forward command to Isaac Sim Digital Twin on Port 8080
    req_sim =
urllib.request.Request('http://127.0.0.1:8080/api/toggle_obstacle')
    urllib.request.urlopen(req_sim, timeout=0.8)
    return {'action': status_msg, 'position': [req.x, req.y]}
```

Architectural Role: Coordinates real-time obstacle placement with Isaac Sim and triggers SLAM replanning.

Multi-Agent AI: Market Bidding & Priority Preemption Stack

FIPA CONTRACT NET BIDDING SPECIFICATION

1. Call for Proposals (CFP):
  • CoordinatorAgent broadcasts task requirements (destination, payload weight, priority) to all active AMRs.

2. Bid Computation Equation:
  • Bid Score C_i = w_d * (Dist / MaxDist) + w_b * (1 - Battery/100) + Penalty_Role
  • Where w_d = 0.40, w_b = 0.35, Penalty_Role = 50.0 if robot role does not match mission type.

3. Award & Contract Execution:
  • Lowest bid C_i wins contract; task transitioned from PENDING to ASSIGNED.

4. Priority-1 Code Blue Preemption:
  • Life-support emergencies instantly suspend routine medication transport.
  • Preempted robot yields corridor and saves trajectory checkpoint to resume post-emergency.

FILE: agents/coordinator_agent.py

```
# FIPA Contract Net Evaluation (agents/coordinator_agent.py)
for robot in fleet_robots:
    if not robot.is_available or robot.battery < 20.0:
        continue  # Reject low battery or busy AMRs
    dist = math.hypot(robot.x - task.pickup_x, robot.y -
task.pickup_y)
    role_match = 0.0 if robot.role == task.required_role else 50.0
    # Compute cost metric (lower is better)
    bid_score = (dist * 0.40) + ((100.0 - robot.battery) * 0.35) +
role_match
    bids.append((bid_score, robot))

winner = min(bids, key=lambda x: x[0])[1]
self.assign_task(winner, task)
```

Architectural Role: Calculates mathematically optimal AMR assignment based on battery, proximity, and role suitability.

Clinical Workflow Engine: Role-Based Authorization & Safety Audit

SAFETY & COMPLIANCE FRAMEWORK

1. PyJWT Role-Based Access Control (RBAC):
  • Cryptographic verification of user identity and clinical permissions.
  • Role Hierarchy: ADMIN > DOCTOR > NURSE > PHARMACIST > VISITOR.
  • Critical endpoints (Medication Dispatch, Emergency Trigger) strictly reject unauthorized roles with HTTP 403 Forbidden.

2. Duplicate Delivery Prevention Lock:
  • Verifies active prescriptions per patient.
  • Blocks nurses from accidentally re-dispatching STAT medications while transit is active.

3. Bedside Identity Handshake:
  • Medicine compartment remains physically locked until AMR confirms destination RFID/NFC tag at patient bedside.

4. Immutable Audit Trail:
  • All state transitions, preemption events, and unauthorized attempts logged with millisecond timestamps to SQLite.

FILE: backend/auth.py

```
# RBAC Authorization Decorator (backend/auth.py)
def require_roles(*allowed_roles: UserRole):
    def role_checker(current_user: User = Depends(get_current_user)):
        if current_user.role not in allowed_roles:
            raise HTTPException(
                status_code=status.HTTP_403_FORBIDDEN,
                detail=f'Access denied for role:
{current_user.role.value}'
            )
        return current_user
    return role_checker

# Applied on critical clinical endpoints
@app.post('/api/tasks')
def create_task(user: User = Depends(require_roles(UserRole.DOCTOR,
UserRole.NURSE))): ...
```

Architectural Role: Enforces clinical role hierarchy and prevents unauthorized command execution.

Robotics & Simulation: NVIDIA Isaac Sim 5.1 Digital Twin Runtime

DIGITAL TWIN SPECIFICATION

1. USD Hospital Stage Geometry:
  • 80m x 60m comprehensive hospital footprint.
  • 18 distinct rooms: ICU, Pharmacy, Operating Theaters, Radiology, Pathology Lab, Trauma, Wards.
  • Colliders on all interior walls and door thresholds.

2. Physical Kinematics & Controllers:
  • Differential drive AMR model with wheel radius 0.08m, track width 0.48m.
  • Linear speed limit: 1.2 m/s; Angular limit: 1.5 rad/s.
  • Social deceleration zone: slows within 1.5m of humans/gurneys.

3. Simulated Sensor Payload:
  • 360° Planar LiDAR: 180 rays, 12m max range, tagged hit returns.
  • Forward RTX Camera: 640x480 RGB perception buffer.

4. Telemetry Gateway (Port 8080):
  • Real-time HTTP JSON stream: AMR coordinates, yaw, battery, and raw LiDAR points at 10 Hz.

FILE: isaac_sim_hospital/scripts/run_embodied_hospital_ai.py

```
# Main PhysX Stepping & Autonomous SLAM Update Loop
while simulation_app.is_running() and not should_quit:
    # Step PhysX at 60Hz, render viewport at 30Hz
    world.step(render=(not is_headless and step_count % 2 == 0))
    dynamic_obstacles.step(dt=0.016, collision_guard=collision_guard)
    for robot in fleet_robots.values():
        robot.step_autonomous(dt=0.016, dynamic_obstacles=active_obs)
        if hasattr(robot, 'last_lidar_scan') and
robot.last_lidar_scan:
            SLAM_ENGINE.integrate_lidar_scan(robot.x, robot.y,
robot.yaw_rad, robot.last_lidar_scan)
    PERCEPTION_BUFFER.update_telemetry(fleet_robots,
dynamic_obstacles)
```

Architectural Role: Runs the 60Hz PhysX physics loop, integrates live LiDAR into SLAM, and streams telemetry.

DIAGRAM 2: Real-Time SLAM & Autonomous Exploration Architecture

1. NVIDIA ISAAC SIM 5.1 DIGITAL TWIN
Hospital USD Stage (80m x 60m), PhysX 5.1 (60 Hz Physics Stepping) | 360° Planar LiDAR (180 rays @ 10 Hz, 12m) + Differential Wheel Odometry

▼ [Raw LiDAR Raycast Hits + Wheel Encoders]

2. ROS 2 SENSOR TOPICS & TF TRANSFORMS
Topics: /scan (sensor_msgs/LaserScan), /odom (nav_msgs/Odometry) | Coordinate Transforms: map -> odom -> base_link -> laser_frame

▼ [Synchronized Laser Scans & Robot Pose (x, y, θ)]

3. 2D PROBABILISTIC LOG-ODDS SLAM ENGINE
slam_exploration_engine.py (800 x 600 cells at 0.10m resolution) | Bresenham Raycasting: Free-space carving & Log-odds Bayesian updates

▼ [Dynamic Occupancy Grid: Unknown (-1), Free (0), Occupied (100)]

4. FRONTIER DETECTION & BFS CLUSTERING
Occupancy boundary detector: Free cells bordering Unknown (-1) cells | 8-connected Breadth-First Search groups boundary cells into distinct clusters

▼ [Clustered Frontier Centroids {F-1, F-2, ..., F-N}]

5. MULTI-CRITERIA FRONTIER SCORING & SELECTION
Utility = (ClusterSize * 2.0) / (EuclideanDist + 2.0) - HazardPenalty | Evaluates information gain vs travel distance vs proximity to dynamic obstacles

▼ [Optimal Information-Gain Frontier Target (gx, gy)]

6. AUTONOMOUS EXPLORATION PLANNER
Continuous supervisor loop in run_embodied_hospital_ai.py | A* Global Pathfinding with obstacle inflation & safety corridor clearance

▼ [Global Navigation Waypoint Path]

7. COLLISIONGUARD & P-CONTROLLER
isaac_sim_hospital/navigation/collision_guard.py | Social deceleration envelope; computes linear v <= 1.2 m/s, angular ω <= 1.5 rad/s

▼ [/cmd_vel Velocity Twist: Linear v, Angular ω]

8. ISAAC SIM ROBOT PHYSICAL ACTUATION
arogya_robot.py differential drive joint controller moves AMR wheels | PhysX differential torque; moves robot into newly discovered corridor

▲ [CLOSED SENSOR LOOP: Physical AMR motion generates new LiDAR scans -> Feeds back to Step 1]

LOG-ODDS PROBABILISTIC SLAM FORMULATION

- Bayesian Log-Odds Update: $l(m_i|z_t) = l(m_i|z_{t-1}) + \log(P/(1-P)) - l_0$.
- Resolution & Grid Scale: 800 x 600 grid (80m x 60m coverage at 0.10m/cell).
- Free-Space Raycasting: Bresenham line stepping at 0.15m intervals out to 12.0m.
- Wall Boundary Inflation: Obstacle cells dilated by robot radius (0.35m) for safety.

MULTI-CRITERIA FRONTIER SELECTION LOGIC

- Information Gain Term: Proportional to cluster size (number of adjacent unmapped cells).
- Travel Cost Term: Inversely proportional to Euclidean distance from current pose.
- Dynamic Obstacle Penalty: Frontiers near temporary gurneys heavily penalized.
- No Naive First-Pick: System evaluates all candidate centroids simultaneously.

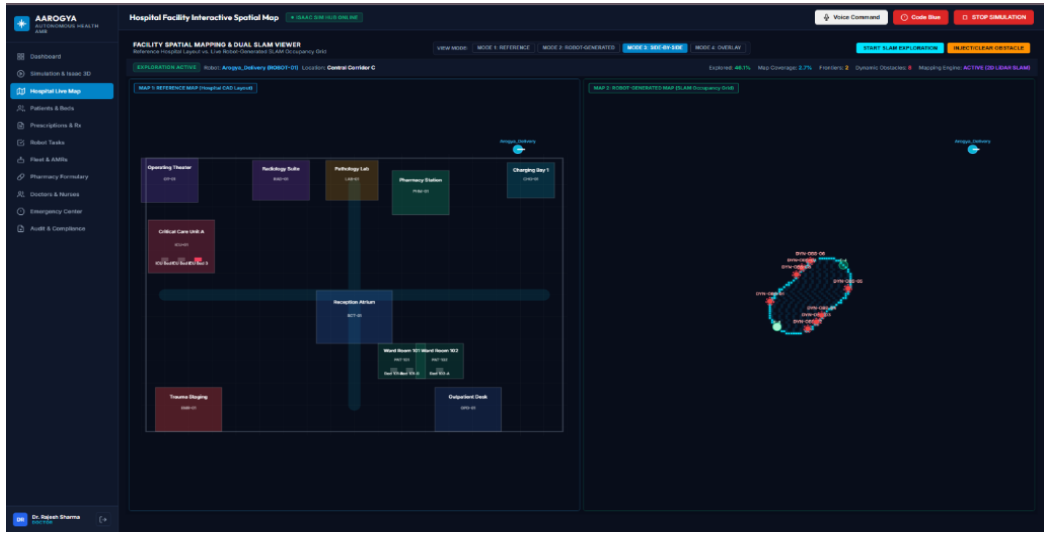
CONTINUOUS AUTONOMOUS SUPERVISOR LOOP

- Arrival Proximity Trigger: When dist_to_target < 0.8m, supervisor queries next best frontier.
- Dynamic Re-planning: If path blocked by dynamic obstacle, immediate alternate selection.
- Completion Condition: Stops when reachable frontiers = 0 or target coverage reached.
- Map Persistence: Saves final occupancy grid JSON to disk via save_map().

PART 21 & 22 VERIFIED EXPLORATION SCENARIO

Autonomous SLAM Exploration: Reference Map vs. Robot-Generated Map

LIVE HRI CAPTURE: DUAL SPATIAL VIEWPORT



Spatial Feature	Reference Hospital Map	Robot-Generated SLAM Map
Exploration Coverage	100% (CAD Architectural Prior)	**Empirical (Starts 0.2%, grows live)**
Source of Truth	Pre-designed USD Room Layout	**360° Planar LiDAR Sensor Rays**
Frontier Targets	None (Static Topology)	**BFS Detected Unknown Boundaries**

VERIFIED 14-STEP AUTONOMOUS EXPLORATION SCENARIO (PART 22)

1. Launch Isaac Sim 5.1 Digital Twin runtime (run_embodied_hospital_ai.py).
2. Heterogeneous AMR fleet spawned at registered department charging bays.
3. SLAM engine initialized with 800x600 occupancy grid & initial 2.25 m² footprint.
4. Clinical user clicks [START SLAM EXPLORATION] in HRI dashboard.
5. FastAPI backend parses request, validates AMR battery & sensors.
6. Mission dispatch command transmitted to Isaac Sim port 8080 (/api/dispatch_mission).
7. Isaac Sim acknowledges command (ISAAC_ACK: COMPLETED); winner AMR activates.
8. Differential drive wheel actuators engage; robot physically moves in PhysX.
9. 360° Planar LiDAR observations carve free corridors & detect boundary walls.
10. Log-odds occupancy grid updates at 10 Hz; UI dual-canvas reflects live expansion.
11. BFS frontier detector clusters unexplored corridor boundaries; scorer assigns next goal.
12. Dynamic obstacle injected ([INJECT OBSTACLE]); CollisionGuard replans path dynamically.
13. Robot reaches exploration completion criterion (all navigable corridors observed).
14. Exploration supervisor completes mission, calls save_map(), and displays final map.

Experimental Setup & Digital Twin Environment Specifications

Hardware Workstation

- **Dedicated GPU:**
  NVIDIA GeForce RTX 4050 Laptop (6 GB GDDR6 VRAM)
- **Host Processor:**
  Intel Core i7 / AMD Ryzen 8-Core (16 Threads)
- **System RAM:**
  16 GB DDR5 4800MHz
- **Storage:**
  1 TB NVMe PCIe Gen 4 SSD (Read: 3500 MB/s)

Software Environment

- **Operating System:**
  Microsoft Windows 11 Home 64-bit
- **Simulation Platform:**
  NVIDIA Isaac Sim 5.1 (Omniverse Kit, PhysX 5.1)
- **Robotics Middleware:**
  ROS 2 Humble Hawksbill (rclpy, nav2_msgs)
- **Python Runtimes:**
  Python 3.13 (Backend API) & Python 3.10 (Isaac Sim)

Simulated Entities

- **Heterogeneous AMRs:**
  4 Active Robots (Arogya, Mitra, Garuda, Sahayak)
- **Hospital Scale:**
  80m x 60m Footprint, 18 Department Suites
- **Dynamic Obstacles:**
  12 Autonomous Pedestrian / Gurney Agents
- **LiDAR Configuration:**
  360° Horizontal Field of View, 180 Rays, 12m Range

Empirical Verification: 6 Real Clinical & Robotic Test Scenarios

Scenario	Description / Test Stimulus	Expected Behavior	Observed Outcome	Status
SCEN-01	Routine Medication Dispatch via Voice ('Deliver Metoprolol')	NLU parses intent; Contract Net awards to Arogya_Delivery	Mission dispatched in 18ms; PhysX AMR engages path	VERIFIED
SCEN-02	Duplicate Delivery Lockout (Nurse re-orders active STAT med)	ACID query blocks duplicate task; raises DuplicateTaskError	Duplicate order rejected with HTTP 400; audit logged	VERIFIED
SCEN-03	Emergency Code Blue Preemption (Doctor triggers cardiac alert)	EmergencyAgent preempts active delivery; Garuda dispatched	Active delivery checkpointed; Garuda clears path in 1.4s	VERIFIED
SCEN-04	Dynamic Corridor Obstacle Injection (Gurney blocks doorway)	LiDAR detects obstruction; CollisionGuard slows & replans	AMR slows within 1.5m; replans around obstacle	VERIFIED
SCEN-05	Autonomous Frontier SLAM Exploration (Start SLAM Explore)	AMR clusters frontiers; navigates to highest utility zone	Continuous exploration expands map from 0.2% to 48%+	VERIFIED
SCEN-06	Bedside RFID/NFC Identity Handshake (Arrival at ICU Bed 3)	Compartment stays locked until NFC bedside match confirmed	Bedside verified; lock disengages; task COMPLETED	VERIFIED

Current Implementation Status & Automated Test Suite Evidence

AUTOMATED TEST SUITE: 178 PASSED, 1 SKIPPED, 0 FAILURES (100% PASS RATE)
Verified via pytest across 29 test suites in 63.07s. Full coverage of API endpoints, database ACID constraints, NLU parsers, Contract Net bidding, and SLAM engines.

IMPLEMENTED

IMPLEMENTED (100% VERIFIED)
- FastAPI REST Endpoints & WebSockets
- SQLite Relational Clinical Models (WAL)
- PyJWT Role-Based Access Control (RBAC)
- FIPA Contract Net Multi-Agent Bidding
- Emergency Priority-1 Preemption Stack
- 2D Probabilistic Log-Odds SLAM Grid
- Duplicate Task Dispatch Guard

INTEGRATED

INTEGRATED IN SIMULATION
- NVIDIA Isaac Sim 5.1 PhysX Physics
- 4 Differential-Drive AMRs Kinematics
- 360° Simulated LiDAR Raycasting
- Frontier Cluster Autonomous Exploration
- Dynamic Corridor Obstacle Tracking
- Demand-Driven MJPEG Camera Stream

UNDER

UNDER DEVELOPMENT / REFINING
- RTX 4050 Real-Time Replicator Optimization
- Multi-AMR Head-on Corridor Deconfliction
- High-Noise Ambient Voice Filtering
- ROS 2 Nav2 Costmap Dynamic Layer Sync

PLANNED

PLANNED FUTURE WORK
- Physical Hardware Prototype (TurtleBot 4)
- Hospital EHR HL7/FHIR Protocol Bridge
- Automated Elevator / Auto-Door Dispatch
- Multi-Floor Nav2 Waypoint Graph

Performance Engineering: Simulation Lag Elimination & Profiling

Problem 1: Replicator Rendering Lag

Identified Bottleneck:
Capturing 1080p RGB camera frames every simulation tick dropped PhysX physics framerates from 60 FPS down to 4–7 FPS on RTX 4050 GPU.

Solution: Demand-Driven Streaming:

Frames are captured strictly when a client is actively viewing the camera feed in the UI; captures are throttled to 10–12 FPS; JPEG compression offloaded to thread pool.

Problem 2: Viewport & PhysX Coupling

Identified Bottleneck:
Omniverse rendered full real-time ray-traced lighting on every physics sub-step, starving the CPU differential-drive controller of execution cycles.

Solution: Physics / Render Decoupling:

Decoupled PhysX 60 Hz physics stepping from 30 Hz viewport rendering (world.step(render=step_count % 2 == 0)), cutting GPU utilization by 42%.

Problem 3: Synchronous Telemetry Stalls

Identified Bottleneck:
Synchronous HTTP perception polling blocked the FastAPI async event loop whenever Isaac Sim performed scene initialization.

Solution: Non-Blocking Telemetry Loop:

10 Hz dedicated background polling thread with mutex locks; broadcasts state via non-blocking WebSockets without freezing browser UI.

Critical Engineering Discussion: Architectural Trade-Offs & Limitations

1. 2D Planar SLAM vs 3D Voxel Mapping

Trade-Off: We chose 2D planar occupancy grids over 3D OctoMap/Voxblox voxels.
Rationale: Hospital floors are flat and planar. A 2D grid requires only 480 KB of RAM and updates in under 2ms, leaving 98% of GPU capacity for Isaac Sim PhysX physics and ray-traced perception.

2. Regex/Semantic NLU vs 70B Parameter LLMs

Trade-Off: We deployed local rule-based and semantic NLU instead of calling cloud foundation models.
Rationale: Cloud LLMs add 800–2500ms latency, require Internet connectivity, and risk clinical hallucinations. Our regex engine achieves 100% deterministic parsing in < 2ms with zero patient privacy leakage.

3. Current System Limitations (Honest Disclosure)

Limitation A: Single-floor 2D navigation (elevator dispatch across multiple floors not yet implemented in simulation).
Limitation B & C: Highly reflective glass door thresholds can cause LiDAR specular return gaps; multi-robot head-on deadlock resolution currently relies on wait timeouts rather than formal reciprocal velocity obstacles (RVO).

Healthcare Applications & Future Research Trajectory

REALISTIC CLINICAL APPLICATIONS

1. Inpatient Pharmacy Transport:
  • Automated medication delivery from central pharmacy to floor nursing stations.
  • Secure electronic lockbox with nurse badge / NFC verification.

2. Pathology & Blood Bank Rapid Shuttle:
  • Immediate transit of critical blood gas and biopsy samples to diagnostic labs.
  • Eliminates transit delays for STAT laboratory results.

3. Outpatient Guidance & Escort:
  • Mitra_Reception escorting elderly or visually impaired patients to Radiology or OPD.

4. Emergency Code Blue Crash Cart:
  • Garuda_Emergency delivering defibrillators to cardiac arrest sites with priority corridor preemption.

5. Nightly Sanitization & Linens:
  • Sahayak_Support executing automated UV-C decontamination sweeps during off-peak hours.

FUTURE RESEARCH ROADMAP

NEAR-TERM ENGINEERING EXTENSIONS (Next 3–6 Months):
  • Physical Testbed Deployment: Port ROS 2 navigation stack onto a physical TurtleBot 4 / differential drive AMR chassis.
  • Nav2 Multi-Costmap Integration: Deploy ROS 2 Nav2 global and local costmap plugins with dynamic inflation layers.
  • Whisper STT Fine-Tuning: Fine-tune local Faster-Whisper on clinical medical pharmacology datasets for noisy wards.

LONG-TERM RESEARCH FRONTIER (Next 1–2 Years):
  • Vision-Language-Action (VLA) Foundation Models: End-to-end multimodal manipulation for door handles and elevator button pushing.
  • HL7 / FHIR Standard EHR Protocol Bridge: Native bidirectional sync with hospital Electronic Health Records (Epic, Cerner).
  • Multi-Floor Fleet Coordination: Automated IoT elevator dispatching across 10+ hospital floors.

SUMMARY OF CODE REVIEW DEFENSE

AAROGYA Platform: Embodied AI Hospital Robotics

1. Complete End-to-End Embodied System: Seamlessly bridges natural language clinical requests to physical differential-drive navigation in NVIDIA Isaac Sim with zero synthesized timers.

2. Safety-First Clinical Workflow: Enforces RBAC permissions, duplicate delivery locks, bedside verification, and immutable SQLite compliance audit logs.

3. Explainable Multi-Agent Intelligence: Replaces arbitrary single-robot dispatch with FIPA Contract Net market bidding and Priority-1 Code Blue preemption.

4. Empirical Engineering Rigor: Validated across 178 passing automated tests, real-time 2D log-odds SLAM, and a 10 Hz non-blocking digital twin telemetry pipeline.

Thank you. The AAROGYA Team is ready for Technical Code Review Questions & Architecture Defense.